Exploratory Testing for Charter III

Session Participants

Eric Deng 261138815 — eric.deng@mail.mcgill.ca (Session lead)

Nathan Audegond 261171251 — nathan.audegond@mail.mcgill.ca (Session participant)

Section 1: Session Summary

1.1: Session Charter

Identify capabilities and areas of potential instability of the "rest api todo list manager". Identify documented and undocumented "rest api todo list manager" capabilities. For each capability, create a script or small program to demonstrate the capability. Exercise each capability identified with data typical to the intended use of the application.

Session focus: Explore /categories, /categories/:id and /categories/:id/todos|projects to discover capabilities and instabilities.

1.2: Version

Release version 1.5.5 of the Todo List Manager, running with profile version=3 (default data).

1.3: Set Up

macOS 26 with Java 25. Requests were issued with curl from the session script so that every request can be replayed exactly; responses were captured in the session log. No Postman collection was used.

1.4: Build

Download from https://github.com/eviltester/thingifier/releases/tag/1.5.5 — runTodoManagerRestAPI-1.5.5.jar, started with: java -jar runTodoManagerRestAPI-1.5.5.jar (API and documentation at http://localhost:4567 and http://localhost:4567/docs).

Section 2: Session Notes

2.1: Session Date and Time

October 14, 2026. Duration: 45 minutes (timeboxed). Requests are numbered by step in Table 2 in the exact order they were run; the raw log preserves that order (curl logs do not carry per-request wall-clock times).

2.2: Exploration Notes

Table 1: Legend for request colours

Legend	
	As Expected
	Bug
	To Explore

Table 2: Session Notes taken during Exploratory Testing

Time	Endpoint Tested	Response	Reference File
Setup	**Run the application through the jar file** java -jar runTodoManagerRestAPI-1.5.5.jar	Application available at http://localhost:4567; default (profile version=3) data present.	runTodoManagerRestAPI-1.5.5.jar (course-provided; not included in this package)
Step 1	**Test API GET request** /categories Documented? DOCUMENTED Expected: 200 with all categories	**Status 200: OK** Observed: HTTP 200 — {"categories":[{"id":"1","title":"Office","description":""},{"id":"2","title":"Home","description":""}]} Result: **As Expected** Side note: List order follows storage, not id order	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 2	**Test API POST request** /categories body: {"title":"errands","description":"outside"} Documented? DOCUMENTED Expected: 201 created with both fields stored	**Status 201: Created** Observed: HTTP 201 — {"id":"219","title":"errands","description":"outside"} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh

Time	Endpoint Tested	Response	Reference File
Step 3	**Test API POST request** /categories body: {"description":"no title"} Documented? DOCUMENTED Expected: 400 "title : field is mandatory"	**Status 400: Bad Request** Observed: HTTP 400 — {"errorMessages":["title : field is mandatory"]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 4	**Test API POST request** /categories body: {"title":""} Documented? DOCUMENTED Expected: 400 "title : can not be empty"	**Status 400: Bad Request** Observed: HTTP 400 — {"errorMessages":["Failed Validation: title : can not be empty"]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 5	**Test API POST request** /categories/1 body: {"description":"amended"} Documented? DOCUMENTED Expected: 200; POST amends only the given field	**Status 200: OK** Observed: HTTP 200 — {"id":"1","title":"Office","description":"amended"} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 6	**Test API PUT request** /categories/1 body: {"title":"Office"} Documented? DOCUMENTED Expected: 200; PUT replaces: omitted description is reset to ""	**Status 200: OK** Observed: HTTP 200 — {"id":"1","title":"Office","description":""} Result: **As Expected** Side note: Replace semantics: the "amended" description from the previous probe is lost	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 7	**Test API GET request** /categories/1/todos Documented? DOCUMENTED Expected: 200 with an empty list (no links yet)	**Status 200: OK** Observed: HTTP 200 — {"todos":[]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 8	**Test API POST request** /categories/1/todos body: {"id":"2"} Documented? DOCUMENTED Expected: 201: link todo 2 to category 1	**Status 201: Created** Observed: HTTP 201 — (empty response body) Result: **As Expected** Side note: Empty response body	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 9	**Test API GET request** /categories/1/todos Documented? DOCUMENTED Expected: 200 listing todo 2	**Status 200: OK** Observed: HTTP 200 — {"todos":[{"id":"2","title":"file paperwork","doneStatus":"false","description":""}]} Result: **As Expected** Side note: Link confirmed from the category side	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 10	**Test API GET request** /todos/2/categories Documented? DOCUMENTED Expected: Category 1 should be listed for todo 2 (relationship is between the two)	**Status 200: OK** Observed: HTTP 200 — {"categories":[]} Result: **Bug** Side note: BUG-02: empty list — the link is one-directional	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 11	**Test API POST request** /categories/1/projects body: {"id":"1"} Documented? DOCUMENTED Expected: 201: link project 1 to category 1	**Status 201: Created** Observed: HTTP 201 — (empty response body) Result: **As Expected** Side note: Empty response body	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 12	**Test API GET request** /categories/1/projects Documented? DOCUMENTED Expected: 200 listing project 1	**Status 200: OK** Observed: HTTP 200 — {"projects":[{"id":"1","title":"Office Work","completed":"false","active":"false","description":"","tasks":[{"id":"405"}]}]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 13	**Test API GET request** /projects/1/categories Documented? DOCUMENTED Expected: Category 1 should be listed for project 1	**Status 200: OK** Observed: HTTP 200 — {"categories":[]} Result: **Bug** Side note: BUG-03 (mirror): empty list	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 14	**Test API DELETE request** /categories/1/todos/2 Documented? DOCUMENTED Expected: 200: relationship deleted	**Status 200: OK** Observed: HTTP 200 — (empty response body) Result: **As Expected** Side note: Empty response body	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 15	**Test API DELETE request** /categories/1/todos/2 (repeated) Documented? DOCUMENTED Expected: 404: relationship already deleted (invalid operation)	**Status 404: Not Found** Observed: HTTP 404 — {"errorMessages":["Could not find any instances with categories/1/todos/2"]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Step 16	**Test API GET request** /categories/987654 Documented? DOCUMENTED Expected: 404 for a missing category	**Status 404: Not Found** Observed: HTTP 404 — {"errorMessages":["Could not find an instance with categories/987654"]} Result: **As Expected**	logs/exploratory_session3_categories.log exploratory/session3_categories.sh
Close	**Review findings and cross-reference bugs**	Findings, concerns, risks and new test ideas reviewed and recorded in Section 4; bugs entered in bug_reports.md with the same BUG numbers used there.	bug_reports.md

Section 3: Referenced Files (Scripts, Programs, Screen Shots, Video Clips, Spreadsheets, etc)

Table 3: Descriptions of Referenced Files and Created Scripts

File Name	Description
exploratory/session3_categories.sh	Curl capability script used in this session: demonstrates every capability exercised, in order. Shared helper: exploratory/common.sh.
logs/exploratory_session3_categories.log	Raw session log: the exact status code and response body of every request in Table 2, in step order. This log is the evidence for the Response column (no screenshots were taken; the log is replayable via the script).
session_notes/CH3_Categories_Notes.pdf	This session-notes document. The teacher-template data is contained in this PDF itself: participant/charter information in Section 1, the per-request method, method type, documented status, expected behaviour, expected-vs-observed verdict, observed behaviour, side notes and XML/evidence details in Section 2, and the summary of findings, concerns and test ideas in Section 4.
bug_reports.md	Consolidated bug form and the full bug reports (summary, description, impact, steps to reproduce) for the bugs in Table 4.
tests/test_categories_api.py, tests/test_category_relationships_api.py, tests/test_bugs_expected_behavior.py, tests/test_bugs_actual_behavior.py	Unit tests created from this session's findings, including the expected-behaviour / actual-behaviour pairs for each numbered bug.
Part_A_Report.pdf	Final Part A report: deliverables, findings, suite structure and repository description.

Section 4: Summary of Session Findings

4.1 Summary of Session Findings

Capabilities identified:

- Category CRUD (title mandatory, description optional) — documented.
- Link / unlink todos and projects from the category side — documented.
- Category links are stored one-directionally (see findings).

What was tried:

Create valid / missing title / empty title; amend with POST; replace with PUT; link todo 2 and project 1 from the category side, then read back from the todo and project sides; unlink twice; GET a missing category.

What we learned:

- Plain category operations are reliable; validation matches todos (title mandatory, non-empty).
- POST amends a single field; PUT replaces and resets the omitted description to "".
- Links created from the category side are NOT visible from the todo or project side, and vice versa (BUG-01/02/03): each direction is stored independently.
- Category <-> project linking works from the category side but is invisible from the project side (BUG-03).

Decisions made:

- Test both directions of every relationship, since "relationship between X and Y" implies symmetry.
- Sequence the probes so each link is created, read from both sides, then deleted in the same session.

Table 4: Descriptions of Identified Bugs

Bug	Description
BUG-02 — Category→todo link is not visible from the todo side	POST /categories/1/todos {"id":"2"} returns 201 and GET /categories/1/todos lists todo 2, but GET /todos/2/categories returns an empty list. The link is stored one-directionally.
BUG-03 — Category→project link is not visible from the project side	POST /categories/1/projects {"id":"1"} returns 201 and the category lists the project, but GET /projects/1/categories returns an empty list. Same one-directional storage as BUG-02 (BUG-01 is the todo-side mirror, found in Charter IV).

4.2 Concerns Identified in Session / Areas of Potential Risk

- BUG-01, BUG-02, BUG-03.
- Inconsistent read model: a client sees different data depending on which side of a relationship it queries.
- PUT replace silently clears fields a client did not intend to change.

4.3 New Testing Ideas Identified in Session

- Category holding 1,000 todos (scale).
- Deleting a category that is linked from both sides.
- Renaming a category and re-reading it from linked todos.
- All category probes repeated with XML payloads.